

Physical AI SDK - Model Lifecycle & Strategy 2026

Hari Krishna Vydana

PAVS AI

Model Lifecycle Overview



Phases of Lifecycle

1. Enablement

- Recieve the model from AIG
- Get the model running
- Off-the-shelf installtion (solve dependencies)
- benchmarking (respective metrics w.r.t.model)
- Identify the right dataset
- Fill the docs with the intial results and expectation

2. Benchmarking

- Measure baseline performance (task-metric, compute-metrics)
- Competitive analysis, With a right dataset on gpu vs cpu & gpu-npu
- Work with CI/CD to automate it

3. Profiling

- Do compute analysis
- Find Hotspots and bottlenecks
- Work with AIG to make profiler as a tool with Physical AI SDK
- KPI re-assesment & updation

4. Fine-tuning

- Expore oppertunities to fix hotspots
- Imlement the Model optimizations
- Benchmark the improvements in docs

Progressive Model Support Strategy

Phase 1: Individual Model/Tool Support

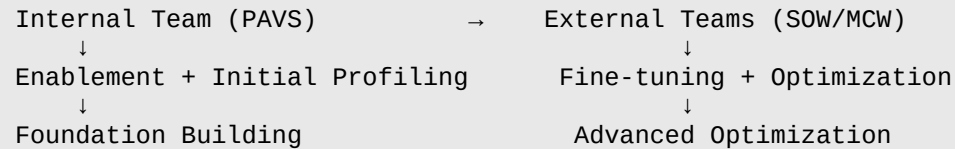
- Focus on single model enablement and optimization
- Establish baseline performance for each model
- Build reusable infrastructure and tooling
 - vLLM, Lemonade,..etc
- Might get simple as we gain expertise

Phase 2: Sample Applications

- Systems: Pipeline of multiple models working together (More complex)
- System-level optimization and resource management will be critical
- End-to-end pipeline integration
- Will give a real feel of product

Effective Team Collaboration

Progressive Engagement Model



Strategy:

PAVS Team: Handle goal setting

- Establish working baseline
- Identify optimization opportunities
- Create clear specifications for external teams

External Partners: Deliverables with goals

- Tangable and measurable outcomes
- More quality time on Improvements
- Domain-specific optimizations

Benefits:

- ✓ Better ROI on external resources
- ✓ Clear handoff criteria
- ✓ Faster time to production

Model Status Across Lifecycle Stages

Model Name	Category	Enablement	Profiling	Benchmarking	Fine-tuning
MobileSAM	Vision/Segmentation	PAVS	MCW	MCW	MCW
YoloV12	Object Detection	PAVS	PAVS	PAVS	MCW
Qwen3-VL	VLM	PAVS	AIG	AIG	AIG
LLaMA3.2-Vision	VLM	PAVS	MCW	MCW	MCW
Whisper	ASR	PAVS	MCW	MCW	MCW
XTTS	TTS	PAVS	PAVS	PAVS	AIG
DeepSeek-R1	LLM	PAVS	MCW	MCW	MCW
Qwen2.5-VL	VLM	PAVS	AIG	AIG	AIG
CrossFormer	Vision Transformer	PAVS	MCW	-	-
CenterPoint	3D Detection	PAVS	-	-	-
OpenVLA	Vision-Language-Action	PAVS	-	-	-
MedSigLIP	Medical Classification	PAVS	-	-	-
EfficientNetV2	Image Classification	-	-	-	-
RAFT Stereo	Stereo Matching	-	-	-	-

Legend: Complete | In-Progress | Not-Started

Key Insight: Start early, execute in parallel across multiple models,

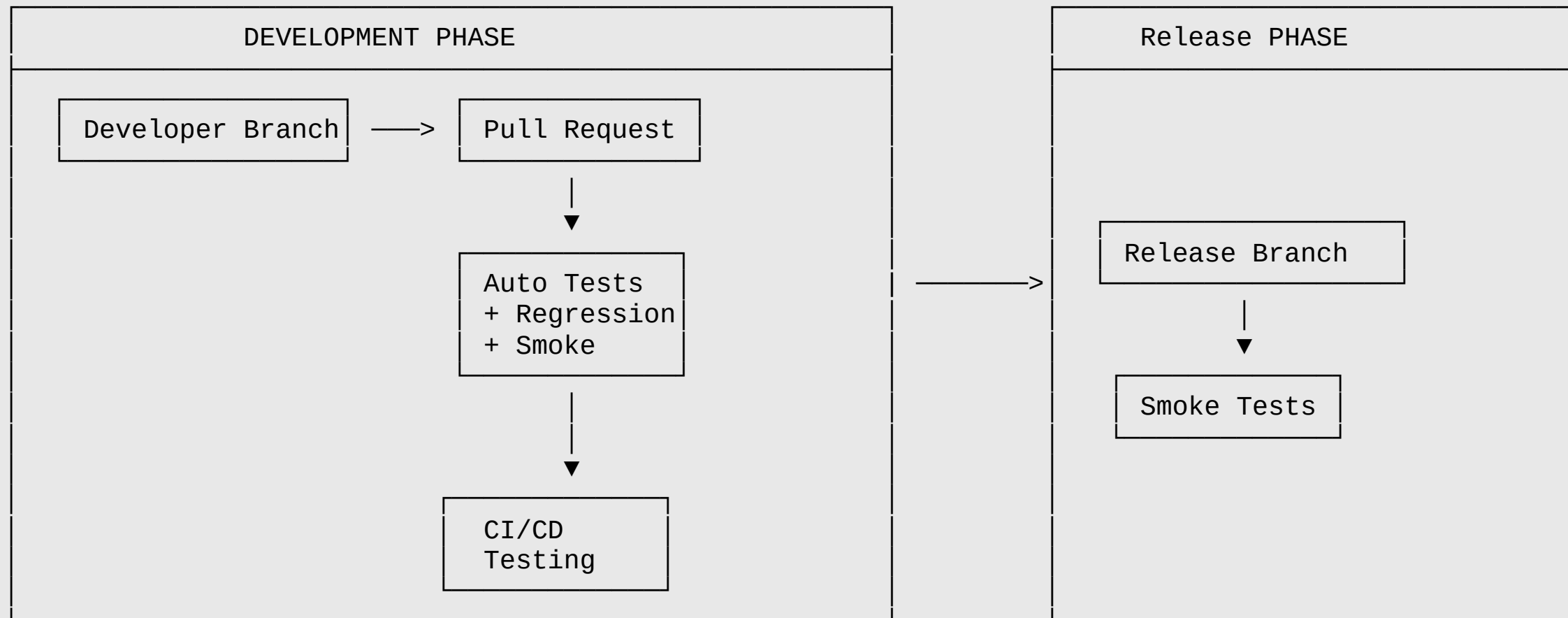
Slide 6: Environment & CI/CD Infrastructure

Environment Strategy:

```
physical-ai-sdk/  
├── models/  
│   ├── mobilesam/  
│   │   ├── gpu/  
│   │   │   ├── scripts/  
│   │   │   ├── inference.py  
│   │   │   ├── Dockerfile  
│   │   │   ├── pyproject.toml (uv)  
│   │   │   └── .venv/  
│   │   ├── npu_gpu/  
│   │   │   ├── scripts/  
│   │   │   ├── inference.py  
│   │   │   ├── Dockerfile  
│   │   │   ├── pyproject.toml (uv)  
│   │   │   └── .venv/  
│   │   └── npu/  
│   │       ├── scripts/  
│   │       ├── inference.py  
│   │       ├── Dockerfile  
│   │       ├── pyproject.toml (uv)  
│   │       └── .venv/  
│   └── yolov12/  
│       ├── gpu/  
│       │   ├── scripts/  
│       │   ├── inference.py  
│       │   ├── Dockerfile  
│       │   ├── requirements.txt (pip)  
│       │   └── .venv/  
│       ├── npu_gpu/  
│       │   ├── scripts/  
│       │   ├── inference.py  
│       │   ├── Dockerfile  
│       │   ├── requirements.txt (pip)  
│       │   └── .venv/  
│       └── npu/  
│           ├── scripts/  
│           ├── inference.py  
│           ├── Dockerfile  
│           ├── pyproject.toml (uv)  
│           └── .venv/  
└──
```

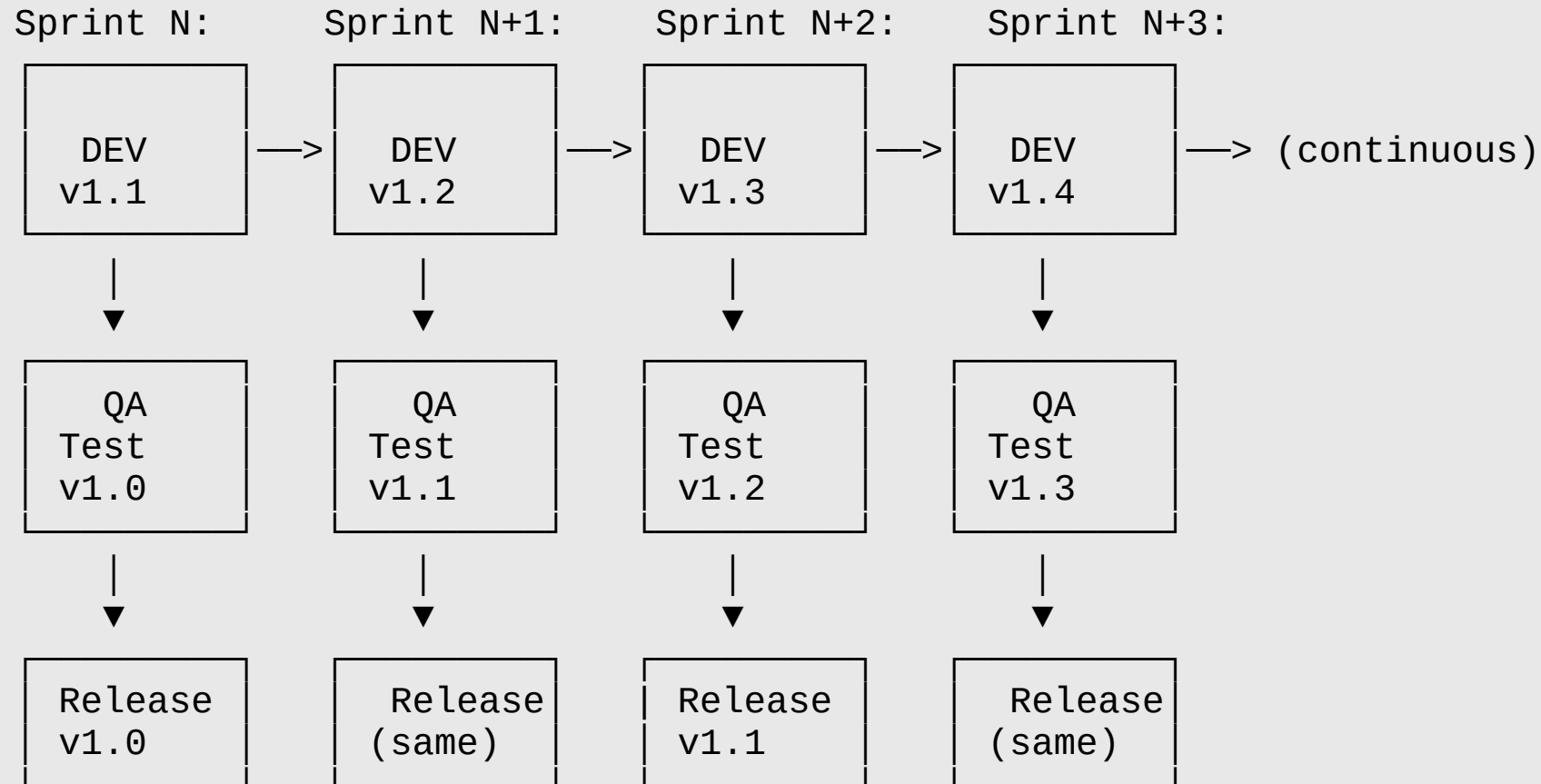
Benefits: ✓ Isolated dependencies | ✓ No version conflicts | ✓ Easy

CI/CD Pipeline:



Dev - Release Timelines:

Sprint Timeline View:



Slide 7: Container & Virtual Environment Options

Option 1: Dockerfile

```
FROM rocm/pytorch:latest
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
CMD ["python", "inference.py"]
```

Option 2: uv + pyproject.toml

```
[project]
name = "mobilesam"
version = "1.0.0"
dependencies = [
    "torch>=2.0.0",
    "onnxruntime-rocm",
]
```

Option 3: pip + requirements

```
torch==2.0.0
onnxruntime-rocm==1.17.0
opencv-python==4.8.0
```

Model Distribution

- 📦 ****HuggingFace Hub**** - Pre-optimized ONNX/TensorRT, Quantized (INT8, FP16), ROCm builds
- 🔗 ****Public Access**** - Anyone can download and test
- ✅ ****Verified Benchmarks**** - Performance metrics included
- 📁 ****Alternative:**** GitHub Releases, Company registry, Cloud storage

Expanding Model Portfolio

Staying Current with Latest Open Source Models

Emerging Neural Alternatives:

-  **End-to-End Neural Pipelines** replacing traditional CV pipelines
 - Example: Neural speech-to-speech model
 - Example: VLA++, pi-0++
-  **Latest Model Categories to Track:**
 - Multimodal Foundation Models (GPT-4V alternatives)
 - Action-VLMs (RT-2, OpenVLA successors)
 - Neural Rendering (NeRF, Gaussian Splatting for robotics)
 - On-device Edge LLMs (< 3B parameters)

Strategy:

1. Monthly scan of arxiv.org, HuggingFace, GitHub trending
2. Quick feasibility assessment (2-3 days cycle)
3. Add to roadmap if aligned with Physical AI use cases
4. If the model fits in requirements and aligns with our business directions (Robotics, healthcare, Industrial we can add it)

Benchmarking & Documentation Strategy

Documentation with Code:

```
docs/
├── index.md
├── models/
│   ├── mobilesam.md
│   ├── yolov12.md
│   └── benchmarks/
├── guides/
│   ├── quick-start.md
│   └── deployment.md
├── tools/
│   └── profilers/
│       ├── roc_profilers.md
│       └── Lemonade_profiler.md
└── _config.yml
```

Automated Workflow:

1. **Code + Benchmarks** → GitHub Repo
2. **CI/CD** → Auto-generate tests, run benchmarks
3. **GitHub Pages** → Publish automatically(Example: [docs](#))

Benefits:

-  Version-controlled documentation
-  Automated benchmark updates
-  Professional presentation
-  Community collaboration

Practical Example - Model Lifecycle

Case Study: Qwen3-VL Integration

Enablement:

-  Model weights download and conversion
-  Basic ONNX-RT integration
-  Simple inference test (single image)
-  Docker environment setup

Profiling:

-  Layer-wise performance analysis
-  Memory usage tracking
-  Hotspot identification (Conv3D, GEMM)

Benchmarking:

-  Performance on dataset vs leaderboard
-  KPI testing: Target 30fps @ 1080p
-  Accuracy validation on VQA datasets
-  Power consumption measurement
-  Automate the benchmarking setup

Fine-tuning:

-  Kernel optimization for GEMM
-  INT8 quantization (W8A8)
-  Automotive dataset fine-tuning

Resource Management Strategy

Balancing Internal & External Resources

Resource Allocation Matrix:

Lifecycle Stage	Internal Team (PAVS)	External Partners	Max.Exptd Duration
Enablement			1 weeks
Profiling			3 days
Benchmarking			3 days
Fine-tuning			2-3 weeks

Handoff Criteria:

- ✓ Model runs successfully on target hardware
- ✓ Profiling report completed
- ✓ Baseline benchmarks established
- ✓ Optimization opportunities documented, explored

Success Metrics & KPIs

Measuring Lifecycle Effectiveness

Model Lifecycle KPIs:

Metric	Target	Current	Trend
Average Time to Enablement	< 2 weeks	1 weeks	⬇️ Improving
Models in Profiling Stage	5-8 concurrent	6	➡️ Stable
Benchmark Pass Rate (1st try)	> 80%	75%	⬆️ Improving
Fine-tuning Speedup (avg)	> 2x	2.3x	⬆️ Good
CI/CD Pipeline Success Rate	> 95%	97%	✅ Excellent

Portfolio Metrics:

- Total Models Supported: **20**
- Production-Ready Models: **15**
- Models in Development: **5**
- Experimental/Research: **5**

Roadmap & Next Steps

R1 2026 (Current):

-  Establish lifecycle framework
-  Complete models lifecycle planing
-  Streamline Release Logistics
-  Deploy GitHub Pages documentation

R2 2026:

-  Implement SDK Installation
 - Make it single installer(Like Nvidia, Intel tools)
-  Expand to 8 release-ready models(P0's and P1's)
-  Implement automated benchmarking CI/CD
-  Implement Quantization - Beta Version
-  Implement Infra - Beta Version (Looks like mostly integration)
 - vLLM, ONNX-RT, Llama.cpp, Lemonade
-  Start sample applications(may need exploration)
 - ChatBot, Obj.detect, opencv/cvml, Gstreamer + Rocm/HIP, robotics simulator

R3 2026:

-  Improve-SDK Installation
-  Expand to 8 release-ready models(P0's and P1's)
-  Improve - Quantization
-  Expand - Infra
 - Model Deploy, Perf. Profile, robotics simulator
-  Expand sample applications
 - Gstreamer+ Ryzen-AI
-  Scale fine-tuning operations
-  Add emerging models from (P2)

R4 2026:

-  Expand to 2 release-ready models(P0's and P1's)
-  Production deployment for 3 segments
-  Expand Infra
 - Model Training(Drafter)
-  Complete Backlogs
-  2027 strategy planning